

Code Reasoning Taxonomy: Details

ANONYMOUS AUTHOR(S)

Variable State Reasoning (A). captures cases where the agent describes the runtime state, initialization, or update of a variable or object. This category indicates that the agent is tracking program execution at the level of concrete or symbolic program states.

A-1: Exact Variable State Identification captures cases where the agent explicitly identifies the runtime state of a variable. The value may be concrete, such as `0`, `None`, or `False`, or structural, such as an empty list or an empty dictionary. This pattern is useful because many execution failures originate from a variable having an unexpected value at a specific execution point, which indicates that agents need to understand the variable state to successfully perform SE task rather than merely being aware of its existence (def or use).

When `alias_refcount` is empty and `alias_map` is empty, `single_alias` returns `False` because the sum is `0`, not `1`.

A-2: Variable State Update describes cases where the agent explains how a variable state is updated, assigned, or even reassigned during execution. In the example below, the agent reasons about how the variable encoding is set to a specific value under certain circumstances. Unlike exact state identification, which focuses on what value a variable has, this pattern focuses on how the variable obtains or changes its state. This pattern is especially important for localization because incorrect updates can trigger later execution failures.

When this is used in `'make_msgid(domain=DNS_NAME)'` in `'message.py'`, and the encoding is set to `'iso-8859-1'`, it fails because those non-ASCII characters can't be encoded...

A-3: Variable State Initialization captures cases where the agent reasons about how an initial or default value affects later execution. This often appears in object initialization or default arguments for methods. In the example below, the agent reasons on how to correctly initiate the shape of a rank-0 array. Initial state reasoning is distinct from general variable state update reasoning because the central issue is the value used before iterative or recursive computation begins.

For a rank-0 array with shape `"()"`, this returns `0`, but it should return `1` since there is one element in a scalar. The fix is to use `functools.reduce(lambda x, y: x * y, shape, 1)`, which provides an initial value of `1`.

A-4: Hypothetical Variable State Assumption captures cases where the agent reasons under an assumed runtime state. The agent may not claim that the state always occurs; instead, it uses a hypothetical condition such as "when `alias_refcount` is empty" in the example below to analyze program behavior. This pattern is common when agents reason about edge cases, failing tests in the bug localization or patch verification stage.

When `alias_refcount` is empty and `alias_map` is empty, `single_alias` returns `False` because the sum is `0`, not `1`.

Call Dependency Reasoning (B). captures cases where the agent describes runtime interactions among functions, methods, constructors, or callbacks. This category goes beyond static call-site identification by explaining how execution moves from one component to another.

B-1: Forward Call Dependency Reasoning captures cases where the agent explains that one method calls another during execution. The reasoning proceeds from caller to callee, often identifying how arguments or state are passed into the invoked function. In the example below, the agent mentions that `alter_unique_together` actively calls `_delete_composed_index`. This pattern is useful for explaining bugs that arise because a caller incorrectly invokes another method.

When `alter_unique_together` calls `_delete_composed_index`, it passes `{"unique": True}` as `constraint_kwargs`.

B-2: Backward Call Dependency Reasoning captures cases where the agent reasons from a callee back to the context in which it is invoked. Instead of starting with the caller, the agent begins with a function or method being executed and explains that it is called by another component. For example, in the following box, the agent reasons that `child_axes` is passively called by `__init__()`.

`child_axes` is set during `__clear()`, which is called from `__init__()`.

Loop Reasoning (C). captures cases where the agent describes repeated execution across loop iterations. This category is distinct from general variable state reasoning because the agent must reason about iteration order, loop-related states, or loop termination.

C-1: Iteration Level Reasoning captures cases where the agent explains what happens during a specific loop iteration. The agent may describe the value of an iteration variable, the behavior of the loop body, or the effect of one iteration on the program state. In the following example, the agent reasons about the state of `item` in the first iteration of a `For` loop. This pattern is useful when a bug occurs only on a particular iteration, which propagates to the later iterations.

In the first iteration, `item` is `""`, so the loop body handles an empty string before processing the remaining items.

C-2: Loop Postcondition Reasoning captures cases where the agent explains the state of the program after a loop finishes. The agent may reason about whether a final update occurred, whether a condition was checked too early or too late, or whether a value remains unchanged after all iterations. This pattern is especially useful for agents to locate if the bug occurs in a loop statement.

When the named group is at the end of the pattern without a trailing `/`, the loop finishes without appending the group to `group_pattern_and_name` because the `if unmatched_open_brackets == 0` check happens at the beginning of the loop iteration.

Conditional Statement Reasoning (D). captures cases where the agent explains how runtime conditions determine the execution path. This category includes predicate evaluation and branch selection.

D-1: Predicate Value Reasoning captures cases where the agent evaluates whether a conditional expression is true or false during execution. The agent may explain the value of a predicate based on variable states, object fields, or previous execution steps. Predicate value reasoning often serves as the basis for explaining why a branch is or is not executed.

On subsequent evaluations with different columns, the condition `if not compiler.query.values_select` is false, so the new values are not applied.

D-2: Branch Selection Reasoning captures cases where the agent identifies which branch of a conditional statement is executed. The focus is on the control-flow decision, such as entering the `if` branch, taking the `else` branch, or skipping a block. This pattern is important because many program behaviors depend on which branch is selected at runtime. The key signal is that the agent explicitly connects a condition to the branch that execution follows.

When there is no existing aggregation and other conditions do not force a subquery, the code goes to the `else` branch, which inlines the annotation.

Observable Behavior Reasoning (E). Observable behavior reasoning captures cases where the agent explains what program execution produces or changes. This includes return values/outputs and exceptions.

E-1: Return Value Reasoning captures cases where the agent states what a function or method returns or outputs under a given execution state. The return value may be a concrete value, such as `False` or `None`, or a symbolic result, such as an unevaluated expression. It is a common form of execution reasoning because many unit tests check return values directly.

The issue is in the `'Interval._complement'` method. Let me look at the flow: 1. When `'a'` contains only symbols (Test 2), `'nums == []'` is `True`, so `'None'` is returned, leading to an unevaluated `'Complement'...`

E-2: Exception or Error Reasoning captures cases where the agent explains how execution leads to an exception (e.g., `TypeError` in the following example), crash, or runtime error. This pattern is different from general return value reasoning because the observable behavior is abnormal termination or an error condition. It is especially common in debugging trajectories where the agent explains why a test fails.

`'decimal.Decimal(value)'` raises `'TypeError'` for dict values, but the except clause only catches `'decimal.InvalidOperation'`. The fix is to also catch `'TypeError'` and `'...`

Based on the code reasoning pattern categorization, we detect code reasoning in agents' thoughts using Algorithm 1. The algorithm takes an agent trajectory as input and iterates through the LLM response at each step (Line 3). For each response, it first checks whether the agent mentions code-related entities, such as variable names, method signatures, or other program elements, which serve as strong indicators of code reasoning (Line 4). If such entities are present, the algorithm further searches for instances of the code reasoning categories and subcategories defined in Figure 4, using regular-expression-based pattern matching. Once any of these patterns is detected in an LLM response, the algorithm marks that step of the trajectory as containing code reasoning.